

Lab 3 — Code Hint Sheet

Fine-Tuning a Pretrained Transformer

Stuck on a **YOUR TURN** cell? Each blank you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the three marked cells need editing; run everything else as-is. `CHECKPOINT`, `NUM_LABELS (=5)`, and `SCRATCH_LR` are already defined for you.

Step 3 — attach a fresh classification head. Load DistilBERT *and* bolt a brand-new 5-class head on top (one head per disease class). This is the whole idea of transfer learning: keep the pretrained body, swap the head (Lecture 8).

Blank 1 — build model.

- (a) `model = AutoModelForSequenceClassification.from_pretrained(CHECKPOINT, num_labels=NUM_LABELS, attn_implementation=ATTN)`
- (b) `model = AutoModel.from_pretrained(CHECKPOINT)`
- (c) `model = AutoModelForSequenceClassification.from_pretrained(CHECKPOINT)`

(a) loads the body *and* a new 5-way head. (b) is the body only — no classifier, so it cannot output class scores. (c) forgets `num_labels`, so it defaults to a **2**-class head and the shape check fails (it wants `[batch, 5]`). The `attn_implementation=ATTN` argument (already defined for you as `"eager"`) picks an attention kernel that runs on a Colab GPU, an Apple-silicon Mac (MPS), and plain CPU alike — the fast default kernel cannot apply attention dropout while training on MPS.

Step 3 — the “frozen body” run. For the head-only comparison, freeze every weight in the DistilBERT *body* so only the new head learns. The body is `model.distilbert`; the head is `model.classifier`.

Blank 2 — freeze the body.

- (a) `for param in model.distilbert.parameters(): param.requires_grad = False`
- (b) `for param in model.parameters(): param.requires_grad = False`
- (c) `for param in model.classifier.parameters(): param.requires_grad = False`

(a) freezes only the body $\Rightarrow$ the check sees exactly **594,437** trainable head params. (b) freezes *everything* — nothing can learn. (c) freezes the head and leaves the body trainable — the opposite of what “head-only” means.

Step 4 — the fine-tuning learning rate. When you start from pretrained weights you nudge them *gently* — a large step would erase what the model already knows. Fine-tuning uses a much smaller rate than the from-scratch run (`SCRATCH_LR = 5e-4`). Lecture 4: the learning rate is the single most important knob.

Blank 3 — FINETUNE_LR.

- (a) `FINETUNE_LR = 2e-5`
- (b) `FINETUNE_LR = 5e-4`
- (c) `FINETUNE_LR = 0.1`

(a) is the usual transformer fine-tuning rate — small, so the pretrained knowledge survives. (b) equals the *from-scratch* rate; the check wants fine-tuning to step more gently than that. (c) is enormous for a pretrained model and blows the weights apart.

*Answers self-check in the notebook: if the `assert` cells pass — the head emits `[batch, 5]`, the frozen run trains exactly **594,437** params while the body’s `requires_grad` is off, and `FINETUNE_LR < SCRATCH_LR` — you chose right. Then watch the three-way bar plot: from-scratch stalls near the **~33%** majority floor while the fine-tuned model reaches the **~60s%** — that gap is the whole point of the lab.*